

QualiJournal Admin — End-to-End 재현 가능한 런북

이 문서는 QualiJournal Admin 패널에서 **QA 자동 점검** 버튼이 동작하지 않거나 컨테이너 배포가 실패하는 문제를 재현-해결하기 위한 단계별 가이드입니다. 세 가지 핵심 문제(토큰 누락, 파이프라인 스크립트 누락, 컨테이너 가져오기 실패)를 해결하고 다시 성공적으로 자동 점검을 수행하는 방법을 정리했습니다.

0. 배경 및 준비

- **관리 토큰 준비:** `ADMIN_TOKEN` 은 비밀키로 저장되어 있으므로 Cloud CLI로 읽어 변수에 보관합니다.
- **환경 변수 설정:** 프로젝트/리전/서비스명을 자신의 환경에 맞게 지정합니다.

```
$PROJECT="quali-journal-prod"
$REG_SEO  ="asia-northeast3"      # 서울(주배포)
$REG_TYO  ="asia-northeast1"      # 도쿄(도메인)
$SRC      ="quali-journal-admin"   # 소스 서비스
$LIVE     ="quali-admin-domap"     # 도메인 서비스
$TOKEN = (gcloud secrets versions access latest --secret=ADMIN_TOKEN --project $PROJECT |
Out-String).Trim()
```

1. 프론트엔드 수정 - 토큰 자동 부착 및 QA 버튼 배선

문제

- 모든 API 요청에 토큰을 붙여야 하지만 기존 `index.html` 은 `Authorization` 헤더만 사용하여 `/api/status` 등에서 401 오류가 발생했습니다.
- `runQa()` 정의보다 `safeWire('btn-qa', 'runQa')` 가 먼저 실행돼 QA 버튼이 무반응이었습니다.

해결

1. **fetch 오버라이드 보강:** `/api/` 로 시작하는 모든 요청에 `Authorization` 과 `X-Admin-Token` 두 헤더를 모두 추가합니다.

이전 보고서에서 제시한 수정안에 따라 `index.html` 내 `fetch` 재정의의 다음과 같이 교체합니다:

```
<script>
(function () {
  const _origFetch = window.fetch.bind(window);
  function getAdminToken() {
    const t = (typeof window.getToken === "function")
      ? window.getToken()
      : (localStorage.getItem("ADMIN_TOKEN") || "");
    return (t || "").trim();
  }
  function makeUrl(input) {
    try {
      const raw = (typeof input === "string") ? input : (input && input.url) || "";

```

```

    return new URL(raw, location.origin);
  } catch (_) {
    return new URL(String(input || ""), location.origin);
  }
}
window.fetch = function (input, init) {
  init = init || {};
  const url = makeUrl(input);
  const isApi = url.pathname && url.pathname.startsWith('/api/');
  const headers = new Headers(init.headers || (input instanceof Request ? input.headers :
undefined) || {});
  const tok = getAdminToken();
  if (isApi && tok) {
    if (!headers.has('Authorization')) headers.set('Authorization', 'Bearer ' + tok);
    if (!headers.has('X-Admin-Token')) headers.set('X-Admin-Token', tok);
  }
  init.headers = headers;
  if (input instanceof Request) {
    input = new Request(url.toString(), init);
    return _origFetch(input);
  }
  return _origFetch(url.toString(), init);
};
})();
</script>

```

1. **authHeaders 함수 보강:** `authHeaders()` 가 두 헤더를 동시에 반환하도록 수정합니다.

```

<script>
function authHeaders(){
  var t=(localStorage.getItem('ADMIN_TOKEN')||'').trim();
  return t ? { Authorization:'Bearer '+t, 'X-Admin-Token': t } : {};
}
</script>

```

1. **QA 버튼 폴백 배선:** `runQa()` 가 로드된 이후에도 버튼을 제대로 연결하도록 `DOMContentLoaded` 이벤트에서 연결하는 폴백을 추가합니다.

```

<script>
document.addEventListener('DOMContentLoaded', function () {
  var btn = document.getElementById('btn-qa');
  if (btn && btn.dataset.wired !== '1') {
    btn.addEventListener('click', function (ev) {
      ev.preventDefault();
      if (typeof window.runQa === 'function') window.runQa();
    }, { once:false });
    btn.dataset.wired = '1';
  }
}

```

```
});  
</script>
```

1. **저장 후** `admin` 디렉터리로 이동하여 두 서비스를 재배포(섹션 3 참고)하고 `https://admin.standardai.co.kr/?v=랜덤` 으로 캐시를 우회해 새 UI를 확인합니다. `ADMIN_TOKEN` 을 입력했을 때 초록색 “인증됨” 배지가 표시되고 KPI 새로고침, Ready 불러오기, QA 자동 점검 버튼이 모두 동작해야 합니다.

2. 백엔드 스크립트 포함 – orchestrator.py 누락 해결

문제

UI의 QA 자동 점검 버튼을 눌러도 로그에 `orchestrator.py not found in image` 오류가 기록되고 파이프라인이 중단되었습니다. 이는 컨테이너 빌드 컨텍스트에 파이프라인 스크립트가 포함되지 않아 생긴 문제입니다.

해결

1. **파이프라인 스크립트 복사:** `orchestrator.py` 와 관련 폴더(`tools/`, `feeds/`, `data/` 등)가 프로젝트 루트에 있을 경우 `admin` 빌드 컨텍스트로 복사합니다. 예:

```
cd "C:\Users\user\Desktop\퀵리저널"  
Copy-Item .\orchestrator.py .\admin\ -Force  
if(Test-Path .\tools){ Copy-Item .\tools .\admin\tools -Recurse -Force }  
if(Test-Path .\feeds){ Copy-Item .\feeds .\admin\feeds -Recurse -Force }  
if(Test-Path .\data ){ Copy-Item .\data .\admin\data -Recurse -Force }
```

1. **.dockerignore/.gcloudignore 확인:** `.dockerignore` 와 `.gcloudignore` 에 HTML과 Python 스크립트가 누락되지 않도록 아래 규칙을 추가합니다. 이는 빌드 컨텍스트에서 HTML 및 스크립트를 제외하지 않도록 보장합니다.

```
!index.html  
!index_lite_black.html  
!**/*.html  
!*.py  
!**/*.py
```

1. **신규 이미지 빌드:** 다음 섹션의 빌드 과정을 따라 새 컨테이너 이미지로 재배포합니다.

3. 컨테이너 이미지 빌드 & Artifact Registry 권한 구성

문제

`gcloud run deploy` 시 **Revision is not ready and cannot serve traffic. Container import failed** 라는 오류가 발생했습니다. 이는 Cloud Run 서비스 계정이 Artifact Registry에 저장된 이미지를 읽을 권한이 없을 때 발생합니다. Stack Overflow에서도 Cloud Run Admin 및 Cloud Run Developer IAM 역할이 더 이상 Artifact Registry 읽기 권한을 자동으로 부여하지 않으므로 서비스 계정에 **Artifact Registry Reader** 역할을 명시적으로 부여해야 함을 지적합니다 ¹.

해결

1. 서비스 계정 확인 및 권한 부여

```
# 런타임 서비스 계정 확인
$SA_SRC = gcloud run services describe $SRC --region $REG_SEO --project $PROJECT --
format="value(spec.template.spec.serviceAccountName)"
$SA_LIVE = gcloud run services describe $LIVE --region $REG_TYO --project $PROJECT --
format="value(spec.template.spec.serviceAccountName)"

# Artifact Registry Reader 및 Secret Manager Accessor 역할 부여
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA_SRC" --
role="roles/artifactregistry.reader"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA_LIVE" --
role="roles/artifactregistry.reader"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA_SRC" --
role="roles/secretmanager.secretAccessor"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA_LIVE" --
role="roles/secretmanager.secretAccessor"

# Cloud Build SA에도 Artifact Registry writer 권한 부여(이미지 푸시용)
$PN = (gcloud projects describe $PROJECT --format="value(projectNumber)")
$CB_SA = "$PN@cloudbuild.gserviceaccount.com"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$CB_SA" --
role="roles/artifactregistry.writer"
```

1. 이미지 빌드 및 Artifact Registry 업로드

Artifact Registry에 리포지토리가 없으면 먼저 생성한 뒤, 소스 디렉터리에서 이미지 빌드 및 푸시를 수행합니다. 예시에서 리포지토리 이름은 `quali` 입니다.

```
cd "C:\Users\user\Desktop\퀄리저널\admin"
# 이미지 빌드 및 푸시
gcloud builds submit --tag asia-northeast3-docker.pkg.dev/$PROJECT/quali/qualijournal-
admin:stable
```

1. 이미지로 배포 및 트래픽 전환

이제 소스 빌드가 아닌 **이미지 배포**를 사용합니다. `--set-secrets` 옵션으로 `ADMIN_TOKEN` 을 비밀에서 주입합니다.

```
$IMG = "asia-northeast3-docker.pkg.dev/$PROJECT/quali/qualijournal-admin:stable"

gcloud run deploy $SRC --image $IMG --region $REG_SEO --project $PROJECT \
--set-secrets ADMIN_TOKEN=ADMIN_TOKEN:latest --allow-unauthenticated

gcloud run deploy $LIVE --image $IMG --region $REG_TYO --project $PROJECT \
--set-secrets ADMIN_TOKEN=ADMIN_TOKEN:latest --allow-unauthenticated
```

최신 리비전에 100% 트래픽 전환

```
gcloud run services update-traffic $SRC --to-latest --region $REG_SEO --project $PROJECT
gcloud run services update-traffic $LIVE --to-latest --region $REG_TYO --project $PROJECT
```

1. 헬스 확인

새 리비전이 활성화되면 다음 커맨드로 상태와 보고서 엔드포인트가 모두 HTTP 200을 반환하는지 확인합니다.

```
$URL = gcloud run services describe $LIVE --region $REG_TYO --project $PROJECT --
format="value(status.url)"

curl.exe -s -D - "$URL/health" | findstr /R "^HTTP/* 200"
curl.exe -s -D - -H "X-Admin-Token: $TOKEN" "$URL/api/status" | findstr /R "^HTTP/* 200"
curl.exe -s -D - -H "X-Admin-Token: $TOKEN" -H "Content-Length: 0" -X POST "$URL/api/report"
| findstr /R "^HTTP/* 200"
```

Content-Length: 0 헤더를 주지 않으면 /api/report 는 411 에러를 반환하므로 반드시 포함해야 합니다.

4. UI 최종 검증

1. **캐시 무시 로드:** `https://admin.standardai.co.kr/?v=<임의숫자>` 로 접속해 브라우저 캐시를 우회합니다.
2. **토큰 인증:** 상단 `ADMIN_TOKEN` 입력 후 **입력** 버튼을 누르면 초록색 “인증됨” 배지가 나타납니다.
3. **Ready 목록 확인:** 왼쪽 날짜(예 `2025-10-24`)와 키워드(예 `J-STD-001`)를 맞추고 **Ready 불러오기**를 클릭하면 준비 목록이 나타납니다.
4. **QA 자동 점검 실행:** 우측 **[QA 자동 점검]** 버튼을 클릭해 모달 로그가 다음 순서로 진행되는지 확인합니다:
`collect → approveTop20 → publish → export(md/csv)`. 실패 시 로그에 나타난 오류 메시지를 기준으로 추가 수정이 필요합니다.

5. 추가 구성 (선택 사항)

- **Cloud Scheduler:** 매일 09:00 KST에 `/api/report` 엔드포인트를 자동 호출해 일일 보고서를 생성하려면 Cloud Scheduler 작업을 설정합니다. 작업 정의에 `X-Admin-Token` 헤더를 포함해야 합니다.
- **지속적인 배포:** GitHub Actions 등 CI/CD 파이프라인에서 빌드 컨텍스트에 스크립트가 포함되는지 확인하고 배포 후 `gcloud run services update-traffic ... --to-latest` 단계를 포함하면 재현 절차를 자동화할 수 있습니다.

결론

이 가이드를 따르면 QualiJournal Admin 서비스의 최신 UI가 정상적으로 배포되고, QA 자동 점검 버튼을 통한 수집 → 승인 → 발행 → Export 파이프라인이 완전하게 작동하게 됩니다. 특히 Cloud Run 서비스 계정에 Artifact Registry Reader 권한을 부여해야 한다는 점을 잊지 마세요 ¹. 파이프라인 스크립트의 누락, 캐시 문제, IAM 권한 문제를 체계적으로 해결하면 동일한 오류가 재발하지 않습니다.

¹ Read access on container images deployed to Cloud Run not working - Stack Overflow

<https://stackoverflow.com/questions/79339148/read-access-on-container-images-deployed-to-cloud-run-not-working>